The FLOPs You Save Are Not the Time You Spend: Sparse Attention in Multi-Agent Communication

Anonymous authors
Paper under double-blind review

Abstract

Attention-based inter-agent communication aggregates over all $N-1$ peers per agent per step, and a substantial line of work proposes sparsifying it to the top-k most relevant peers. The efficiency case for doing so is almost always made by counting operations rather than by measuring time. We show that the count does not survive contact with the hardware, and that it could not have delivered much even if it had.

We contribute, first, an analytic ceiling: because selecting the top-k peers *by attention score* requires computing all N scores, only the value aggregation is sparsifiable, and the achievable reduction in attention FLOPs is bounded above by $2\times$ for any k and any N — not the $1-k/(N-1)$ that is usually reported. Structural sparsity, whose pattern is fixed before any score is computed, escapes this bound; data-dependent top-k cannot.

Second, we measure what actually executes. We find that a widely reproduced implementation pattern applies top-k as a *mask* on the dense attention matrix and then runs the same dense aggregation matmul, so its executed multiply–accumulate count is identical to dense attention and the reported saving is never performed. We implement the genuinely sparse gather that the claim is about and benchmark all three variants — dense, masked, gathered — against a fused SDPA baseline. Across a sweep of head width, batch size, agent count and sparsity level, the gathered implementation is slower than dense in 46 of 46 cells, on GPU and CPU, in `fp32` and `bf16`, eager, compiled and CUDA-graph captured. A component decomposition and a roofline analysis explain why: top-k selection alone costs $15.8\times$ the entire dense aggregation at N=512, the sparse contraction is itself slower than the dense one it replaces, and the sparse path moves $3.8\times$ the memory traffic for $0.6\times$ the arithmetic. Both paths are memory-bound, and removing arithmetic from a memory-bound kernel does not save time. We also find that a learned per-agent gate costs $\mathcal{O}(BN^3)$ activation memory (fitted exponent 3.04), exhausting a 96 GiB device at N=465.

Third, a pre-registered training study on a communication-critical task confirms that the channel carries information, while six pre-registered sparsification contrasts are underpowered nulls at five seeds and are reported as such. Finally, an audit of the papers this literature cites for its efficiency argument finds that 1 of 6 multi-agent communication papers reports any measured temporal quantity for its mechanism, against 5 of 6 in the efficient-attention and sparse mixture-of-experts work the argument is borrowed from. We release the benchmark harness, the corrected implementation, and five reproducible benchmarking pitfalls that produced spurious results during this study.

1 Introduction

Cooperative multi-agent reinforcement learning (MARL) systems increasingly pass learned messages between agents, and attention is the dominant mechanism for deciding what to listen to (Das et al., 2019; Jiang & Lu, 2018; Jiang et al., 2018). Because every agent attends to every other, the channel costs $\mathcal{O}(N^2)$ per step, and a natural proposal follows: restrict each agent to its top-k peers, with $k \ll N$. The argument is intuitive, and it is nearly always supported the same way — by an operation count. If dense aggregation touches $N-1$ peers and sparse aggregation touches k, then a $1-k/(N-1)$ fraction of the work disappears.

This paper asks whether that fraction is real. The answer is no, in two distinct senses, and the gap between them is the point.

The count is smaller than advertised. Selecting the top-k peers *by attention score* requires computing every score. The score matrix is therefore irreducible, and only the value aggregation — half of attention — can be sparsified at all. Section 3 states this as a proposition: the achievable reduction in attention FLOPs is bounded above by $2\times$, for every k and every N, approached only as $k/N \to 0$. The familiar $1 - k/(N-1)$ describes one of two equal terms and overstates the achievable saving by a factor approaching two. The bound has a precise escape route, which we make explicit because it is the design target for any successor method: if the sparsity pattern is *structural* — fixed by a topology, a spatial prior, or a hash computable in $o(N^2 d)$ — the score matmul becomes sparsifiable too and the ratio reaches N/k, unbounded. This is exactly why Longformer and BigBird obtain asymptotic savings where data-dependent top-k cannot (Beltagy et al., 2020; Zaheer et al., 2020).

Even the reduced count is not a saving in time. A count is a model of cost, not a measurement of it. We find that the implementation pattern in common use does not execute the saving at all: top-k is applied as a *mask* on the dense attention matrix, the discarded entries are materialised as zeros, and the same dense $(B, N, N) \times (B, N, d)$ aggregation matmul runs. Executed multiply–accumulates are identical to dense. We implement the genuinely sparse gather the claim refers to, verify it computes the same function to floating-point tolerance while executing measurably fewer FLOPs, and benchmark dense, masked and gathered as three separate curves against a fused SDPA baseline. The gathered implementation is slower than dense everywhere we look: 46 of 46 cells in a head-width $\times$ batch $\times$ N $\times$ k sweep, on two hardware architectures, at two precisions, under three execution modes.

Why, mechanistically. Section 4 decomposes the sparse path stage by stage. Two facts account for the result. Top-k selection alone costs $15.8\times$ the entire dense aggregation at N=512; and the sparse contraction is *itself* $3.4\times$ slower than the dense matmul it replaces, despite performing four times fewer multiply–accumulates, because a batched matrix–vector product has far lower arithmetic intensity than a real GEMM. A roofline analysis with empirically measured machine balance shows both paths sitting far below the ridge point on GPU and CPU alike: both are memory-bound, the sparse path moves $3.8\times$ the traffic for $0.6\times$ the arithmetic, and removing arithmetic from a memory-bound kernel does not make it faster. This is also why the result reproduces on a CPU whose machine balance differs from the GPU's by more than an order of magnitude — it follows from the ratio of the two paths' intensities, which is a property of the algorithm.

Contributions.

- A $2\times$ ceiling on the FLOPs saving achievable by any score-based top-k attention, stated with its assumptions separated and its escape route made explicit (Section 3).
- A three-way decomposition of the implementation space — dense, masked, gathered — showing that the masked variant in common use executes the dense operation count, and a corrected sparse implementation that does not (Sections 3–4).
- A measurement campaign over $N \leq 512$, head width 16–256, batch 32–4096, two dtypes, three execution modes and two architectures, finding no crossover anywhere, with a component decomposition and roofline account of the cause, and a drift control quantifying reproducibility (Section 4).
- The finding that a learned per-agent gate costs $\mathcal{O}(BN^3)$ activation memory — derived, and validated at a fitted exponent of 3.04 (Section 4).
- A pre-registered training study separating "the channel does nothing" from "sparsifying the channel is free", with a positive control that passes and six sparsification contrasts reported as the underpowered nulls they are (Section 5).
- An audit of measured-latency reporting across the literature this argument is drawn from, and five reproducible benchmarking pitfalls, each of which produced a spurious result during this study (Sections 2 and 6).

Table 1: Does the paper measure what it claims to save? "Reports latency" requires a measured temporal quantity attributable to the mechanism. The comparison group is the efficient-attention and sparse mixture-of-experts literature from which the MARL complexity argument is borrowed.

Paper	**Efficiency claim**	**Type**	**Reports latency**
Multi-agent communication / attention			
CommNet (Sukhbaatar et al., 2016)	none explicit	—	no
TarMAC (Das et al., 2019)	compact messages	qualitative	no
IC3Net (Singh et al., 2018)	fewer messages	counts	no
ATOC (Jiang & Lu, 2018)	bounded group size	counts	no
DGN (Jiang et al., 2018)	neighbourhood scaling	asymptotic	no
MAT (Wen et al., 2022)	linear in N	asymptotic	**yes**
Efficient attention / sparse mixture-of-experts			
Reformer (Kitaev et al., 2020)	$\mathcal{O}(L \log L)$	asymptotic	**yes**
Longformer (Beltagy et al., 2020)	$\mathcal{O}(L)$	asymptotic	**yes**
BigBird (Zaheer et al., 2020)	$\mathcal{O}(L)$	asymptotic	no
Performer (Choromanski et al., 2020)	$\mathcal{O}(L)$	asymptotic	**yes**
Switch (Fedus et al., 2022)	FLOPs-matched	FLOPs	**yes**
Sparse MoE (Shazeer et al., 2017)	large capacity, low cost	FLOPs	**yes**

Scope. Every claim here concerns *centralised-inference computational cost*: the wall-clock and memory of executing the communication module on one device. It is explicitly *not* a claim about bandwidth-constrained distributed deployment, where the objective is to reduce the number of messages physically transmitted and where top-k sparsity may well be the correct design. Section 7 states the full envelope.

2 Related Work: Who Measures, and Who Counts

Attention-based inter-agent communication. CommNet (Sukhbaatar et al., 2016) broadcasts mean-pooled hidden states; TarMAC (Das et al., 2019) introduces signature-based targeting so that receivers weight senders; IC3Net (Singh et al., 2018) gates whether an agent speaks at all; ATOC (Jiang & Lu, 2018) forms bounded communication groups; DGN (Jiang et al., 2018) restricts attention to a spatial neighbourhood; MAT (Wen et al., 2022) casts the joint policy as a sequence model. These works motivate restricted communication with a cost argument — bandwidth, complexity, or "receiving a large amount of information … incurs high computational complexity" (Jiang et al., 2018).

The audit. We checked whether that argument is ever accompanied by a measurement. For each paper we read the full text and recorded (i) whether it makes a computational-efficiency or complexity claim about its mechanism and (ii) whether it reports any measured temporal quantity — time, speed, throughput, or achieved FLOP/s — attributable to that mechanism. A statement such as "training took a few days" does not qualify and is recorded as not qualifying. Table 1 reports the result.

One of six multi-agent communication papers reports a measured temporal quantity for its mechanism, against five of six in the literature the argument is imported from. We stress what this table is and is not. It is an audit of *reporting practice*, not of correctness: a "no" means the efficiency argument rests on operation counts or asymptotics alone, not that the argument is wrong. It also has an obvious selection caveat — these are the papers this line of work cites, not a random sample of MARL.

The asymmetry is nonetheless informative, because the borrowed argument does not transfer cleanly. The efficient-attention results hold at sequence lengths of 10^3–10^5, where attention is genuinely the bottleneck; MARL communication operates at $N \lesssim 10^2$, where it is not. And the borrowed methods are structurally sparse, so they escape the ceiling of Section 3 that data-dependent top-k is subject to.

Measurement discipline in the efficient-attention literature. One precedent deserves particular mention because it anticipates the distinction this paper turns on. Longformer (Beltagy et al., 2020) reports

runtime and memory for *three* implementations of the same attention pattern: a loop version, a vectorised "chunks" version, and a custom CUDA kernel. The authors note that the chunked variant consumes roughly twice the optimal memory because it "computes some zero values" — that is, they measured a masked implementation separately from a genuinely sparse one, and reported the difference. FlashAttention (Dao et al., 2022) and memory-efficient attention (Rabe & Staats, 2021) make the same point from the other direction: the winning implementations of dense attention reduce memory traffic without changing the operation count at all, which is precisely the axis a FLOPs-based argument cannot see.

Sparse and budgeted attention mechanisms. Sparsemax (Martins & Astudillo, 2016) and α-entmax (Peters et al., 2019) obtain exact zeros from the normalisation itself rather than a hard cutoff; Gumbel-softmax (Jang et al., 2017) provides the relaxation used by learned discrete gates; constrained-RL formulations (Achiam et al., 2017; Stooke et al., 2020) supply the dual machinery for enforcing a communication budget rather than trading it against return at a fixed weight. We implement entmax and a Lagrangian budget as arms in Section 5.

Empirical rigour in MARL. Our training protocol follows the recommendations of Papoudakis et al. (2021) for MARL benchmarking, and we report effect sizes and intervals rather than bare means. Our environments come from the PettingZoo/MPE family (Terry et al., 2020), and our learner is MAPPO (Yu et al., 2022).

3 What Sparsifying Attention Can and Cannot Save

3.1 Setting and three implementations of one function

Consider one round of scaled dot-product attention over N agents, per batch element, with head width d. Agent i forms a query q_i, and every agent contributes a key and a value; self-attention is masked. Write $s_i = q_i K^\top \in \mathbb{R}^N$ for the score vector and $a_i = \mathrm{softmax}(s_i)$ for the attention weights. Dense attention computes $m_i = a_i V$.

A top-k variant keeps only the k largest entries of a_i and renormalises. Crucially, the *same function* admits implementations with very different costs, and the literature does not consistently distinguish them.

masked. Compute a_i in full, multiply by a $\{0,1\}$ mask, renormalise, and evaluate $m_i = \tilde{a}_i V$ with $\tilde{a}_i$ still dense in memory. The discarded entries are materialised as zeros. Executed multiply–accumulates are *identical* to dense, plus the cost of selection and masking.

gathered. Take the top-k of the raw scores, softmax over those k values only, gather the corresponding value rows into a (B, N, k, d) tensor, and contract $(B{\cdot}N, 1, k) \times (B{\cdot}N, k, d)$. No zeros are materialised and the aggregation genuinely shrinks.

Fused dense. Compute the same function as dense through a fused kernel (Dao et al., 2022; Rabe & Staats, 2021) that never materialises the (B, N, N) weight matrix at all.

GATHERED and MASKED compute the same function. Two facts make this so: softmax is monotone within a row, so the top-k weights are the top-k scores; and a softmax over just those k scores equals the dense softmax restricted to them and renormalised. We verify agreement to $< 10^{-10}$ in `float64` and pin it with a regression test. **The efficiency claim in the literature is a claim about gathered**; benchmarking MASKED measures an implementation artifact, and benchmarking against unfused dense flatters the sparse method by ignoring what production kernels actually cost. We therefore report all three throughout.

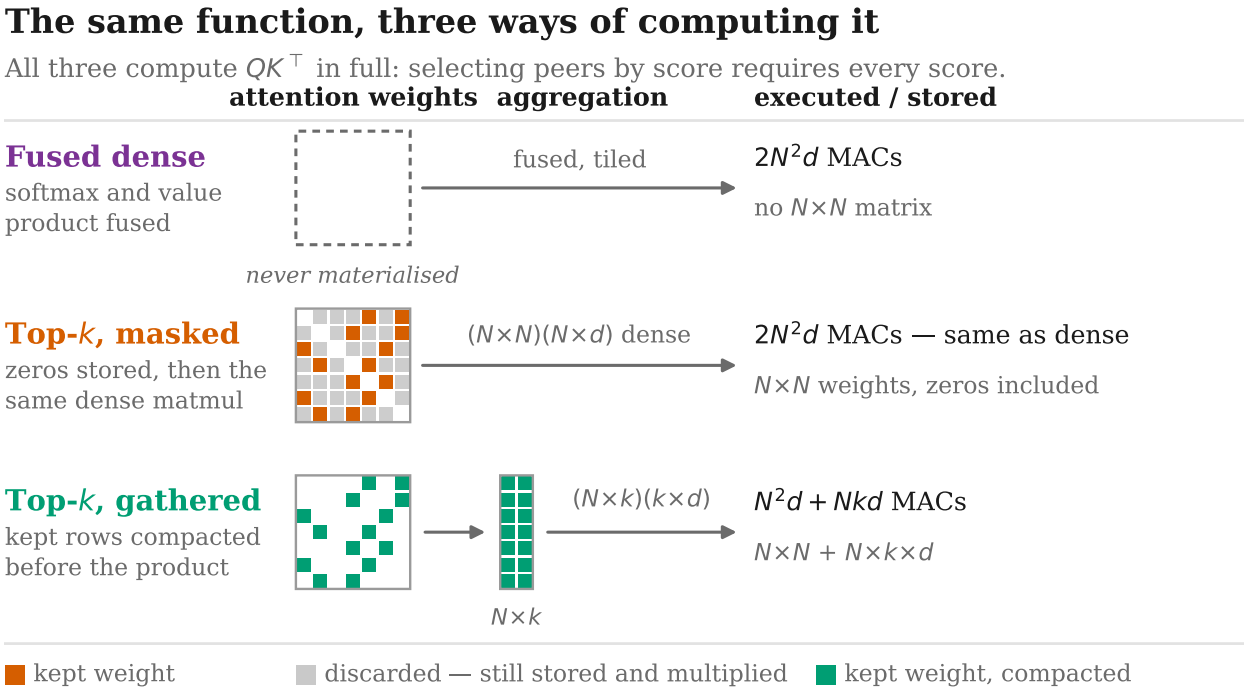

Figure 1: The same top-k attention, computed three ways. All three form the full score matrix, because selecting peers by score requires every score. They differ in what happens next: the fused kernel never materialises the weights; the masked variant stores the discarded entries as zeros and multiplies them anyway, so its executed operation count equals dense; only the gathered variant compacts the kept rows and shrinks the product. **The efficiency claim in the literature is a claim about the third row.**

3.2 A ceiling on what sparsity can save

Counting multiply–accumulates in the two matrix products that constitute attention, and excluding the shared input and output projections:

$$C_{\text{scores}} = N^2 d, \quad C_{\text{agg}} = N^2 d, \quad C_{\text{dense}} = 2N^2 d, \qquad C_{\text{top-}k} = \underbrace{N^2 d}_{\text{all scores}} + \underbrace{Nkd}_{\text{aggregate } k}. \tag{1}$$

Proposition 1 (Ceiling). *For any selection rule that requires the complete score vector of each query,*

$$\frac{C_{dense}}{C_{top\text{-}k}} = \frac{2N^2 d}{N^2 d + Nkd} = \frac{2N}{N+k} < 2 \qquad \textit{for all } k \geq 1, \tag{2}$$

with supremum 2 approached only as $k/N \to 0$. No choice of k or N yields more than a $2\times$ reduction in attention FLOPs.

Proof. $C_{\text{top-}k} > N^2 d = \frac{1}{2} C_{\text{dense}}$ for every $k \geq 1$ since $Nkd > 0$. The ratio decreases monotonically in k and tends to 2 as $k/N \to 0$. □

The commonly reported figure $1 - k/(N-1)$ describes the aggregation term alone, which is at most half of attention; quoting it as *the* saving overstates the achievable reduction by a factor approaching two, before any question of whether it is realised in time.

The assumptions are worth separating, because only one of them is load-bearing. **(A1) Single round**: with R rounds both terms scale by R and the ratio is unchanged. **(A2) Data-dependent selection**: the kept

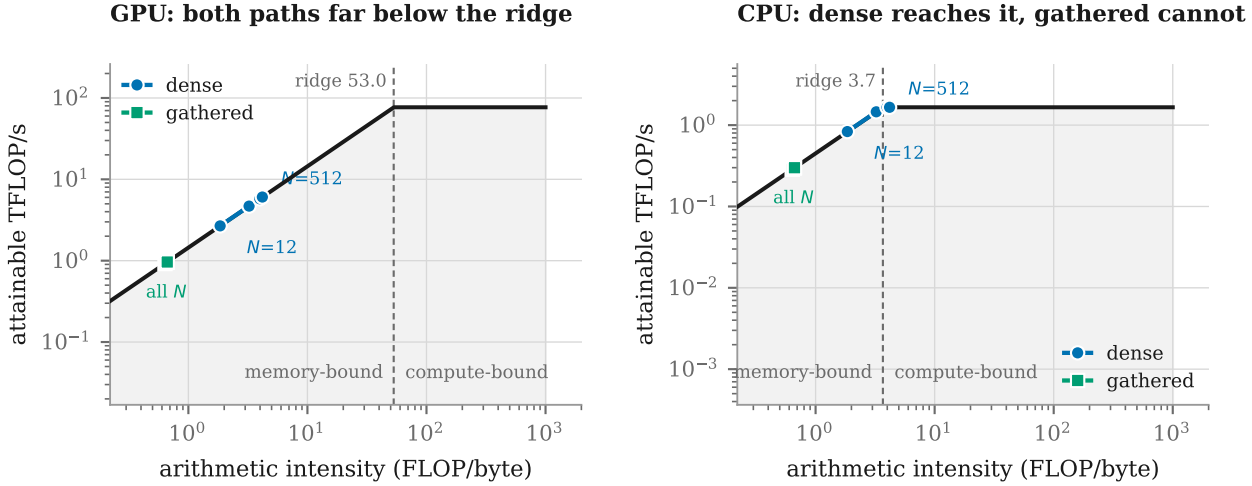

Figure 2: Roofline with *measured* machine balance. On the GPU both paths sit far below the ridge, so both are memory-bound and removing arithmetic cannot buy time. On the CPU — whose balance is an order of magnitude lower — the dense path reaches the ridge while the gathered path, pinned at intensity 0.66 for every N, does not. That asymmetry is why the gap is *wider* on CPU, not narrower.

set is a function of the scores, so all N scores must be formed. **(A3) Per head**: with H heads of width d/H both terms scale identically. **(A4) Projections excluded**: including the shared qkv and output projections moves the ratio strictly closer to 1, so Proposition 1 remains a valid upper bound. **(A5) Selection cost excluded**: counting the top-k operation itself can only decrease the ratio — and Section 4.4 shows this term dominates in practice.

Only A2 lifts the ceiling when it fails.

Proposition 2 (Escape)**.** *If the kept set is determined* structurally *— by information obtainable without forming the scores, in $o(N^2 d)$ — then scores need not be computed for discarded pairs, and $C_{\text{struct}} = Nkd + Nkd = 2Nkd$, giving $C_{dense}/C_{\text{struct}} = N/k$, which is unbounded in N/k.*

Qualifying rules include a fixed communication topology, a spatial or distance prior, a locality-sensitive hash, and a learned *static* graph. This is why Longformer and BigBird obtain asymptotic savings where data-dependent top-k cannot, and it is the concrete design target for any successor method: **to beat 2×, a method must decide where to look before looking.** In an environment with spatial structure, a distance-calibrated selector is the obvious candidate, and we record it as the primary future direction.

3.3 Why the ceiling is not the binding constraint

Proposition 1 bounds a quantity that, in this regime, does not govern runtime. Attention here is memory-bound, and for memory-bound kernels time tracks bytes moved, not arithmetic performed (Williams et al., 2009).

We compute arithmetic intensity AI = FLOPs/bytes analytically per stage, counting each tensor read and written once, and compare it against a *measured* machine balance: a large GEMM gives achievable FLOP/s, a large device-to-device copy gives achievable bandwidth, and their ratio is the ridge point above which a kernel is compute-bound. Measuring rather than quoting datasheet peaks keeps the comparison honest about the specific device and dtype.

Table 2 and Figure 2 give the result. The gathered path adds two stages that move substantial memory for *zero* arithmetic — the top-k selection and the gather — so its intensity is pinned near 0.66 and does not improve with N. At N=512 the sparse path moves 4.16 GB to perform 2.76 GFLOP, against dense's 1.11 GB for 4.63 GFLOP: 3.8× the traffic for 0.6× the arithmetic. Removing multiply–accumulates from a memory-bound kernel does not make it faster, and adding traffic to one makes it slower.

Table 2: Measured machine balance, and analytic arithmetic intensity of the two paths at batch 256, d=16, $k = 25\%$ of peers. Both paths sit far below both ridge points, i.e. both are memory-bound; the sparse path is *further* below.

Device	**Achievable GEMM**	**Achievable bandwidth**	**Ridge point**
RTX PRO 6000 Blackwell (`fp32`)	76.8 TFLOP/s	1448.8 GB/s	53.0 FLOP/byte
x86_64 CPU (`fp32`)	1.66 TFLOP/s	450.8 GB/s	3.68 FLOP/byte
N	**dense AI**	**gathered AI**	**ratio**
12	1.85	0.64	0.34×
48	3.23	0.66	0.20×
192	3.98	0.66	0.17×
512	4.18	0.66	0.16×

This also predicts that the result should be architecture-independent, since it follows from the ratio of the two paths' intensities rather than from any device constant. Section 4 confirms it on a CPU whose machine balance differs from the GPU's by more than an order of magnitude.

4 Measuring the Cost

4.1 Protocol

We time the communication module in isolation, with the training loop removed. The protocol is stated in full because two of its provisions were forced by defects that produced spurious results first (Section 6).

Timing uses on-device CUDA events with an explicit synchronisation on the correct device. Any diagnostic that calls `.item()` is disabled, because such a call synchronises the device *inside* the timed forward pass. Arms are timed **interleaved repetition by repetition** rather than in sequential per-arm blocks, so that any interference from other tenants lands on all arms at once and the ratios between them remain paired; on this shared host the sequential design produced swings above 100% between identical configurations, which interleaving reduced to typically under 2%. We report min over samples as the primary statistic, since contention can only add time, and give median, IQR and inter-run spread for every headline cell in Appendix A.1. GPU clocks could not be locked — the account lacks the privilege — and we say so rather than implying a controlled clock. Executed FLOPs are counted with `torch.utils.flop_counter` so that latency is always reported beside the arithmetic actually performed.

Unless stated otherwise: PyTorch 2.11 / CUDA 12.8, `fp32`, input embedding width 64, head width d=16, batch 256, $k = 25\%$ of peers, and an idle device.

4.2 Three curves

Table 3 is the central result. The gathered implementation does execute fewer multiply–accumulates than dense — 3553 against 5167 MFLOP at N=512, a 31% reduction consistent with Proposition 1 — and it is nonetheless 3.53× slower than unfused dense and 16.29× slower than the fused kernel. Correcting the implementation matters and is not sufficient: replacing MASKED with GATHERED improves the ratio from $\approx 1.45\times$ to $\approx 1.23\times$ at small N, a real gain that does not reach a crossover.

Robustness. The ordering is unchanged under `torch.compile` and CUDA-graph capture (both within 10% of eager), at batch 4096 where the largest cells become FLOP-bound, and in `bf16` where all four fused SDPA backends including FlashAttention are reachable — in `bf16` the gap is *wider*, reaching 19.7× the fused kernel at N=512. On CPU the ordering also holds, up to 37.9× the fused baseline, which is the cross-architecture evidence Section 3.3 predicts.

Table 3: Forward latency (ms, min over interleaved samples) of three implementations of the same function, plus the fused dense baseline. The gathered path executes fewer FLOPs than dense at every N and is slower at every N. Ratios are gathered relative to each dense baseline.

		dense		**top-k**		**ratio**		**MFLOP**
N	k	fused	unfused	MASKED	GATHERED	/unfused	/fused	gath. vs dense
6	1	0.117	0.124	0.174	0.146	1.17	1.24	11 vs 11
12	2	0.117	0.125	0.179	0.151	1.21	1.29	22 vs 23
24	5	0.099	0.120	0.173	0.147	1.22	1.49	47 vs 50
48	11	0.108	0.128	0.184	0.158	1.23	1.46	105 vs 120
96	23	0.105	0.124	0.214	0.224	1.81	2.13	257 vs 315
192	47	0.149	0.206	0.681	0.771	3.75	5.19	703 vs 931
384	95	0.295	1.160	3.499	3.174	2.74	10.78	2161 vs 3070
512	127	0.431	1.989	7.557	7.025	3.53	16.29	3553 vs 5167

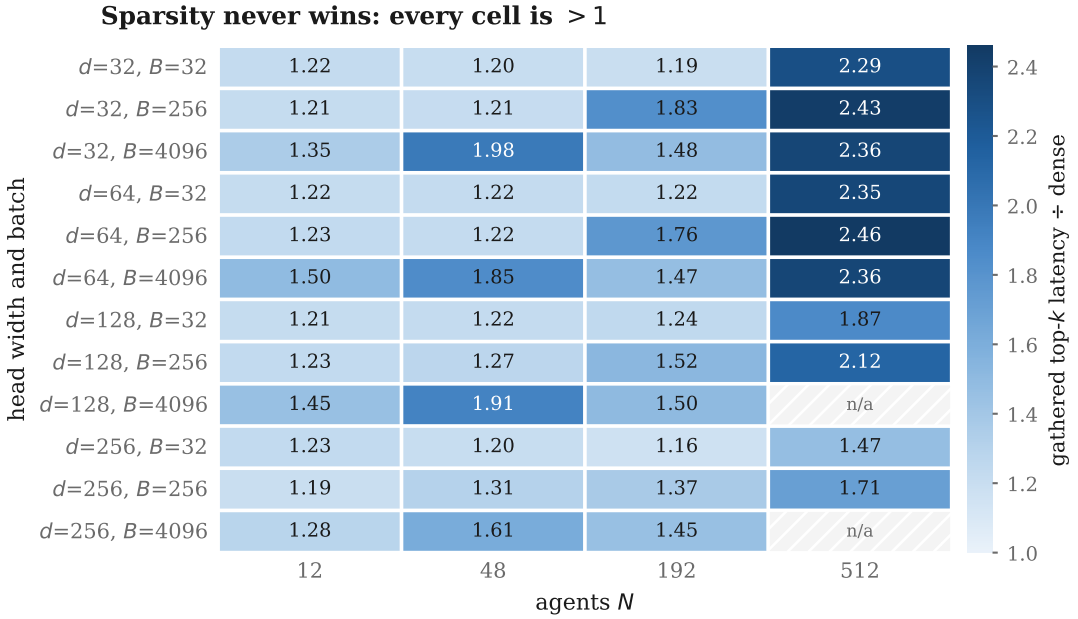

Figure 3: The crossover surface. For every (d, B, N) the *best* k for the gather path is shown, i.e. the most favourable case sparsity can construct for itself. Every cell exceeds 1, and the ratio stays within 1.16–2.46× across an eightfold range of head width and a 128-fold range of batch — the flatness is the finding. Hatched cells exceeded the memory budget.

4.3 Is there any (d, B, N, k) where sparsity wins?

Reporting isolated favourable cells as anomalies is not a characterisation. We swept head width $d \in \{32, 64, 128, 256\}$ against batch $\in \{32, 256, 4096\}$ and $N \in \{12, 48, 192, 512\}$, and for each cell took the *best* k for the gather path — the most favourable case sparsity can construct for itself.

Gathering beats unfused dense in 0 of 46 measured cells (the underlying numbers are tabulated in Appendix A.2), and the fused baseline in 1 of 46 (at d=128, B=4096, N=12, where the fused kernel's fixed overhead dominates at tiny N). The ratio stays within 1.16–2.46× across an eightfold range of head width and a 128-fold range of batch. That flatness is itself the finding: the outcome is structural rather than a consequence of any particular operating point.

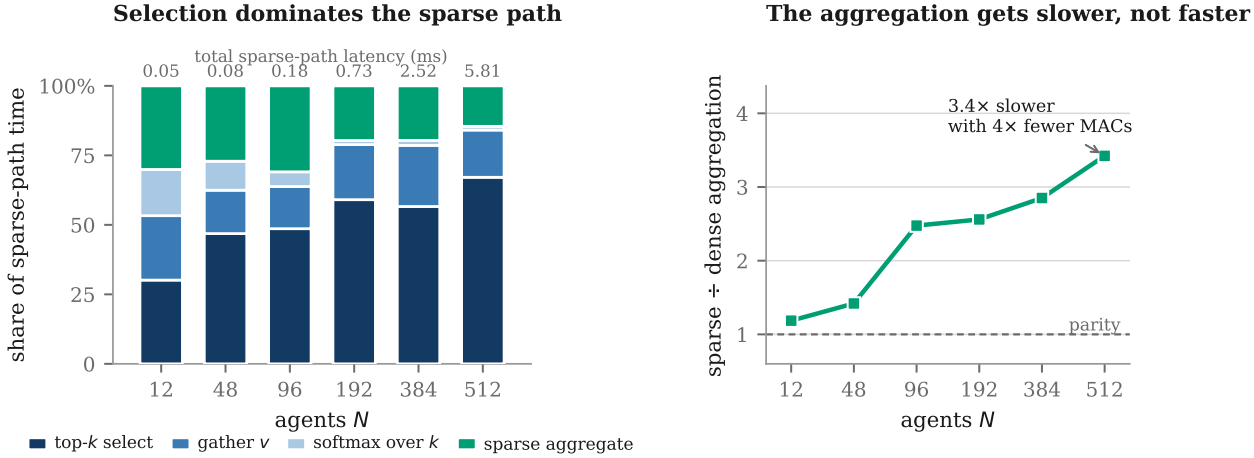

Figure 4: **Left:** how the gathered path spends its time, as a share of its own total (absolute totals above each bar, in ms). Selection grows from 30% to 67% of the budget as N rises. **Right:** the aggregation that sparsity was supposed to shrink. The sparse contraction is slower than the dense matmul it replaces at every N, reaching 3.4× at N=512 while performing roughly four times fewer multiply–accumulates.

Table 4: Per-stage latency (ms). The aggregation "saving" is *negative* at every N: the sparse contraction is slower than the dense matmul it replaces, despite performing ≈ 4× fewer multiply–accumulates. Selection overhead is an order of magnitude larger still.

N	k	`topk`	`gather`	`agg_dense`	`agg_sparse`	saving	overhead
12	2	0.016	0.013	0.014	0.016	−0.003	+0.028
48	11	0.037	0.012	0.015	0.021	−0.006	+0.047
96	23	0.087	0.027	0.022	0.055	−0.033	+0.114
192	47	0.434	0.146	0.056	0.144	−0.088	+0.554
384	95	1.427	0.554	0.174	0.495	−0.321	+1.824
512	127	3.897	0.986	0.247	0.845	−0.598	+4.595

4.4 Where the time goes

Table 3 says *that* sparsity loses; the following says *why*. We time each stage in isolation under the same protocol and form two quantities: the *saving* the sparse contraction buys (`agg_dense` − `agg_sparse`) and the *overhead* selection costs ($\texttt{topk} + \texttt{gather} + \texttt{softmax}_k - \texttt{softmax}_N$).

Figure 4 and Table 4 agree on two facts, and both are stronger than the hypothesis they were meant to test. First, **the aggregation saving is negative**: the sparse $(B{\cdot}N, 1, k) \times (B{\cdot}N, k, d)$ contraction is 3.4× *slower* than the dense $(B, N, N) \times (B, N, d)$ matmul it replaces at N=512, because a batched matrix–vector product has far lower arithmetic intensity than a real GEMM and cannot use the same hardware paths. Second, **selection dominates everything**: `topk` alone costs 15.8× the entire dense aggregation at N=512.

The consequence is that the loss is not an engineering defect a better kernel would remove. It is intrinsic to ranking all N scores — exactly the assumption A2 that Proposition 1 rests on.

4.5 The learned gate costs $\mathcal{O}(BN^3)$ memory

A per-agent learned gate over $k \in \{1, \ldots, N-1\}$, implemented in the natural way, broadcasts a $(1, 1, K, N)$ cumulative-keep tensor against $(B, N, K, 1)$ gate probabilities with $K = N-1$, materialising $(B, N, N-1, N)$ before reducing. Activation memory is therefore $\mathcal{O}(BN^3)$ against dense attention's $\mathcal{O}(BN^2)$ and the fused kernel's $\mathcal{O}(BNd)$.

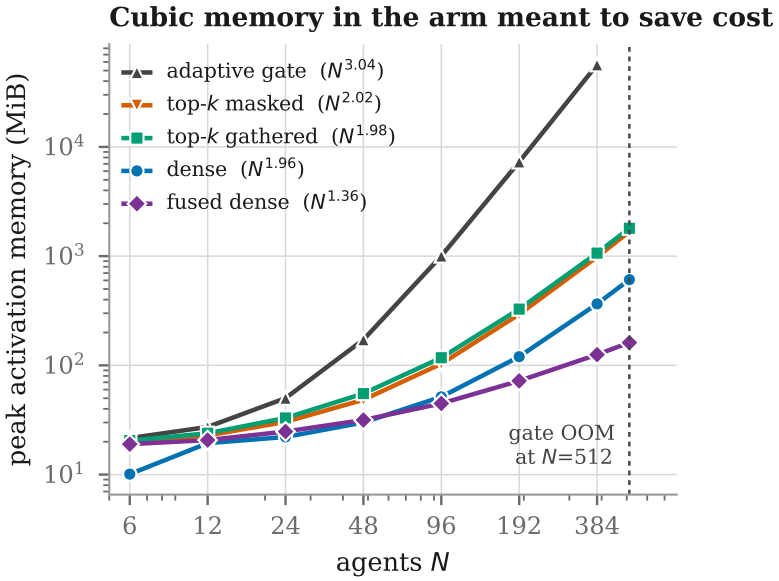

Figure 5: Peak activation memory against agent count, batch 256, with the offset-corrected fitted exponent beside each arm. The learned gate grows cubically and is the first arm to exhaust the device; gathering saves no memory over masking, because the gathered value tensor replaces the zeros it avoids.

Table 5: Reproducibility of the headline sweep. Ratios — the quantity every claim is stated in — are far more stable than absolute latency.

	latency $\lvert\Delta\rvert$		**ratio** $\lvert\Delta\rvert$	
Condition	median	max	median	max
Idle second device, same model	1.9%	6.5%	**1.0%**	**3.2%**
Same device, concurrent training load	41.3%	80.6%	25.9%	56.4%

We validate the derivation rather than asserting it. Fitting peak $\sim N^p$ over $N \geq 48$ with a constant offset removed gives $p = 3.04$ for the gate, against 1.96 for dense, 2.02 for masked, 1.98 for gathered and 1.36 for fused. The sharper check is absolute: at N=384 the closed form predicts 97.5% of the measured peak. In practice the gate consumes 56.5 GiB at N=384 (batch 256) and exhausts the device at N=512; analytically it exhausts a 96 GiB card at N=465, and at batch 4096 at N=185. The arm whose purpose is to reduce cost is the first to run out of memory.

4.6 Drift control

Because clocks are unlocked and the host is shared, we re-ran the identical sweep ≈ 21 h later, twice.

On an idle second device the ratios reproduce to within 3.2% at every N, so the measurement is a property of the algorithm and architecture rather than of one physical card. Under concurrent load agreement degrades substantially, concentrated at small and moderate N where kernels are launch-bound; the largest cells move by about 1%. We state the consequence plainly: **latency must be measured on an unloaded device**, and interleaving plus minimum-over-samples reduces contention sensitivity without eliminating it.

The direction matters more than the magnitude. *Every* drift moves in the direction that makes the sparse path look worse, and the smallest gathered/dense ratio observed anywhere — across all runs, devices, loads, precisions and execution modes — is 1.14×. Contention cannot be concealing a crossover; it can only exaggerate a gap that is already present.

5 Does Sparsification Cost Return?

Sparsification could still be worthwhile if it were free in return. Testing that requires care, because on a fully observable task "sparsification is free" and "the channel was never used" make identical predictions. We therefore gate the study on a positive control and pre-register the analysis.

5.1 Design

Twelve arms, $N \leq 12$, five seeds, 2M environment steps each, in four priority tiers. All N-scaling is delegated to the training-free measurement of Section 4, both because end-to-end wall-clock in this pipeline is environment-bound (throughput differs by $< 3\%$ across arms at every $N \geq 6$, so it cannot attribute a difference to the module) and because a trained N sweep was projected at 995 GPU-hours against a 200-hour budget.

Communication-critical controls. We use two, because neither suffices alone. `simple_reference` is logically unsolvable without the channel — each agent can observe only its partner's goal — but is fixed at N=2, so it cannot carry the k arms. We therefore also introduce a partially observable `simple_spread`: for each agent i and peer j, if $\|\text{peer\_rel}[i,j]\|_2 > R$ then that entry is zeroed. Nothing else changes — reward, action space, landmark observations, episode length and observation dimension are all untouched. We use R=1.0 throughout.

Because this is a custom variant, we characterise it rather than asking for trust. Under a uniform random policy the mean fraction of peers visible at R=1.0 is $0.342, 0.307, 0.263, 0.235$ for $N = 3, 6, 9, 12$, while the absolute number of visible peers rises $0.68 \to 1.54 \to 2.10 \to 2.58$. The two move in opposite directions because `simple_spread` does not grow its world with N, so density rises and a fixed radius hides a different fraction at each N. **This is a difficulty confound across N, and it is why no claim in this paper compares returns across N in this environment.** A per-N calibrated radius would remove it and is the right design for a future N study.

5.2 Pre-registration

The analysis plan was written and committed when 8 of 40 runs had completed and before any tier-2–4 result was inspected. It fixes: the primary endpoint (final greedy evaluation return, one number per seed); the six contrasts and their direction; the test (exact two-sided permutation on the difference of means, enumerating all $\binom{10}{5} = 252$ assignments); the effect sizes (Hodges–Lehmann, Cohen's d); the interval (percentile bootstrap, flagged as approximate at n=5); and the multiplicity correction (Holm–Bonferroni over the six, FWER 0.05).

It also fixes the power expectation *in advance*: the smallest attainable two-sided p at 5 vs 5 is $2/252 = 0.0079$, against a strictest Holm threshold of $0.05/6 = 0.0083$. At most one contrast in the family can reach corrected significance, and only on perfect separation. Any null is therefore under-powered rather than evidence of absence, and no equivalence claim is available because no TOST margin was pre-registered.

5.3 Results

The channel works. Both positive controls separate: dense attention-communication beats communication-free MAPPO by $+4.26$ $[+3.15, +5.23]$ on `simple_reference` and $+11.59$ $[+7.13, +16.44]$ on the partially observable task, at the smallest p the design can produce. Measured attention entropy on the N=6 control is 0.35 nats against a $\ln 5 = 1.61$ ceiling, so attention is strongly peaked rather than uniform. Sparsification is therefore being applied to a channel that demonstrably carries information — the precondition that makes the rest interpretable.

Everything else is an under-powered null. All six pre-registered contrasts return no separation, with Holm-adjusted $p \geq 0.38$. This is the outcome the pre-registered power calculation predicted, and we report it as under-powered, not as "no effect".

Table 6: Tier-1 positive control (inspected before pre-registration; excluded from the correction) and the six pre-registered contrasts. All six return no separation. HL is Hodges–Lehmann.

	Contrast	$A - B$	HL	d	p	p_{Holm}
Positive control						
G1	dense − MAPPO, `simple_reference`	+4.26	—	—	0.008	—
G2	dense − MAPPO, PO `simple_spread` N=6	+11.59	—	—	0.008	—
Pre-registered family						
C1*	gathered k=2 − random k=2	+4.03	+4.57	+0.60	0.357	1.000
C2	gathered k=2 − dense	−0.61	−1.98	−0.12	0.865	1.000
C3	gathered k=4 − gathered k=1	+1.76	+1.21	+0.26	0.659	1.000
C4	gathered k=3 − dense, N=12	+147.01	+122.94	+1.27	0.064	0.381
C5	Lagrangian budget − dense	−6.74	−3.39	−0.93	0.183	0.730
C6	entmax − dense	−5.56	−5.44	−1.13	0.119	0.595

The primary contrast C1 is the one the design exists for: `gathered` and `random-`k are matched on k, on executed FLOPs and on every hyperparameter, differing only in whether peers are chosen by attention score or uniformly at random. Its point estimate favours attention (+4.03, HL +4.57, d = +0.60) but does not approach significance. We can say the design lacked power to detect a selection effect of this size; we cannot say attention selects arbitrarily.

Two honest caveats. C4's sign is the opposite of the expected direction — at N=12 the sparse arm's point estimate is *better* than dense by +147, with dense far more variable, consistent with the gate-variance failure mode reported for learned gates — but it does not reach significance and we do not report it as a finding. And for C4 and C6 the bootstrap interval excludes zero while the permutation test does not; the pre-registration named the permutation test as the test, so it governs, and we record the tension rather than resolving it toward whichever statistic reads better.

Mechanism, descriptive only. The Lagrangian arm meets its budget: expected k settles at 1.16 against a target of 1.25, with the dual multiplier decaying to zero once the constraint is satisfied. Entmax settles at a support of 1.64 of 5 peers without any explicit budget. Attention entropy falls from 0.35 (dense) to 0.12 (k=2) to 0.00 (k=1), as it must. These inform interpretation; they support no claim.

6 Five Pitfalls in Attention Benchmarking

Each of the following produced a wrong number during this study before it was caught. We report them as technical content for anyone benchmarking attention variants, with the measured magnitude of each error, because a benchmarking result is only as trustworthy as the harness that produced it.

P1. A diagnostic `.item()` inside the timed region. The module computed a mean-attention-entropy statistic in its forward pass. The call synchronises the device *inside* the region under measurement, so the benchmark measures the synchronisation, not the module. Symptom: dense attention latency appeared *flat in* N — 0.20 ms at N=6 rising only to 0.48 ms at N=192 — which we initially and wrongly read as evidence that the $\mathcal{O}(N^2)$ cost does not manifest. Fix: gate all diagnostics behind a flag that is off while timing.

P2. Timing arms in sequential blocks on a shared device. Running all repetitions of arm A and then all of arm B makes the comparison hostage to whatever else used the device during each block. Symptom: swings above 100% between identical configurations of the same script, and in one case a 3.7× discrepancy for the same arm at the same N. Fix: interleave arms repetition by repetition so interference lands on all of them simultaneously, and report ratios, which are paired, alongside absolute latency, which is not. This reduced observed spread from $> 100\%$ to typically $< 2\%$.

P3. `torch.cuda.synchronize()` without a device argument. The no-argument form synchronises the *current* device. Timing work submitted to a non-default device therefore waits on the wrong queue. Symptom: every kernel, at every size, reports the CUDA-event resolution floor of $\approx 0.4\,\mu$s — a result that looks like a spectacular speedup rather than an obvious error. Fix: pin the device and pass it explicitly to every synchronisation.

P4. Benchmarking a masked implementation as though it were sparse. The distinction of Section 3.1. Symptom: the "sparse" arm is uniformly slower than dense, which is correct for that implementation but says nothing about the method, since MASKED executes every multiply–accumulate dense does plus selection on top. A FLOPs counter based on $1 - k/(N-1)$ will happily report a 91% saving for code whose executed operation count is identical to dense. Fix: assert that executed FLOPs differ between the two paths, and that the sparse path issues no $(B, N, N) \times (B, N, d)$ product. We pin both as regression tests.

P5. Comparing against an unfused dense baseline. If the deployed alternative is a fused kernel, an unfused reference flatters the sparse method — by 4.6× at N=512 in our setting. Note also that the fused path is invisible to standard FLOP counters, which report zero for it, so a FLOPs-only comparison cannot see the baseline that matters. Fix: report both, and check which SDPA backend actually serves the shape and dtype in question (in `fp32` only the memory-efficient kernel was available to us; FlashAttention requires half precision).

A general form. P1 and P3 are instrumentation errors that inflate or flatten the measurement; P2 is a design error that lets a shared environment leak into the comparison; P4 and P5 are specification errors about *what is being compared to what*. The last two are the more dangerous, because the resulting numbers are internally consistent and reproducible — they are simply answers to a different question than the one the paper asks. The defence is to state the operation count each arm executes alongside its latency, which makes a mismatch between the claimed and the executed algorithm visible immediately.

7 Limitations

What the claim covers. Every cost result here concerns **centralised-inference computational cost**: the wall-clock latency and memory of executing the communication module on a single device, where all agents' observations are available to one process. It is *not* a claim about bandwidth-constrained distributed deployment. In that regime the objective is different — reducing the number of messages physically transmitted over a link — and top-k sparsification may well be the correct design even though it does not reduce local compute. Conflating the two would misrepresent this result, and the analytic ceiling of Proposition 1 says nothing about transmitted bytes.

Measurement envelope. Single-round attention; scaled dot-product attention with the fused backends available in PyTorch 2.11 / CUDA 12.8; input embedding width 64; head width $d \in \{16, 32, 64, 128, 256\}$; $N \leq 512$; batch 32–4096; `fp32` and `bf16`; eager, `torch.compile` and CUDA-graph execution. Hardware: one GPU architecture (NVIDIA Blackwell, sm_120) and one x86_64 CPU. We make no claim outside this envelope.

No cross-generation GPU replication. The strongest available robustness evidence is the CPU result, whose machine balance differs from the GPU's by more than an order of magnitude, plus an idle second device of the same model. We were unable to replicate on a different GPU generation — the older hardware that produced this project's earlier runs is no longer available. "Structurally" versus "on this architecture" is therefore supported by the roofline argument and the CPU agreement rather than by a second GPU generation, and a reader who weights that evidence differently should discount the generality accordingly.

Unlocked clocks. We lacked the privilege to lock GPU clocks. We compensate with interleaved timing, minimum-over-samples reporting, published dispersion (Appendix A.1) and an explicit drift control (Section 4.6), and we report that absolute latency under concurrent load can move by tens of percent. Ratios are far more stable, and all claims are stated in ratios.

Statistical power of the training study. Five seeds per arm gives a minimum attainable two-sided permutation p of 0.0079 against a Holm threshold of 0.0083 for six contrasts. This is enough to detect only perfect separation, and it is a consequence of the compute budget, not a property of the methods. The six null contrasts of Section 5 should be read as "this design could not resolve an effect of this size", and no equivalence between sparse and dense is claimed anywhere. A study powered to resolve the $d \approx 0.6$ effect suggested by C1 would need roughly 45 seeds per arm.

Environment. The partially observable `simple_spread` variant is our own, documented in Section 5 and released. It carries a difficulty confound across N that we measure and publish, and which is why no across-N return comparison appears in this paper. Results on MPE tasks may not transfer to environments with different observation structure.

Implementation scope. GATHERED is a straightforward PyTorch implementation using `gather` and a batched contraction; it is not a hand-written fused kernel. A custom kernel that fuses selection, gather and contraction would reduce the memory traffic that Section 3.3 identifies as the binding constraint, and could plausibly close part of the gap. It could not, however, escape Proposition 1, nor avoid ranking all N scores. We regard such a kernel as the most informative single follow-up to this work, together with the structural-sparsity direction of Proposition 2.

8 Conclusion

The efficiency case for sparsifying inter-agent attention rests on an operation count. We examined that count and then measured what runs.

The count is smaller than reported: because selecting peers by attention score requires every score, only half of attention is sparsifiable and the achievable FLOPs reduction is bounded by $2\times$, not by $1 - k/(N-1)$. The count is also, in the implementation pattern we found in use, not executed at all — masking the dense attention matrix leaves the dense aggregation matmul in place. And when we implemented the sparse gather the claim actually refers to, verified it computes the same function, and measured it against a fused dense baseline, it was slower in every one of 46 cells spanning head width, batch, agent count and sparsity level, on two architectures, at two precisions, under three execution modes. Selection alone costs an order of magnitude more than the aggregation it is meant to shrink, the sparse contraction is itself slower than the dense one, and both paths are memory-bound — so removing arithmetic was never going to help.

None of this shows that sparse multi-agent communication is a bad idea. It shows that the standard argument for it does not survive measurement in the centralised-inference regime, and it identifies precisely what a successor method must do differently. Proposition 2 is the constructive half of the result: a method whose sparsity pattern is fixed *before* the scores are computed — by topology, by a spatial prior, by a hash — is not subject to the ceiling and can reach N/k. In an environment with spatial structure, distance-calibrated routing is the natural instance, and it is where we would spend the next unit of effort. A fused kernel that eliminates the intermediate traffic between selection, gather and contraction is the other.

The wider lesson is methodological, and it is not specific to MARL. An operation count is a model of cost. Like any model it can be wrong in two ways: it can mis-estimate the quantity it describes, and the quantity it describes can fail to govern the outcome anyone cares about. Both happened here. The remedy is cheap — report executed operation counts beside measured latency, so that a divergence between the algorithm claimed and the algorithm executed becomes visible immediately. Our audit suggests the surrounding literature has largely not been doing this: one of six multi-agent communication papers reports any measured temporal quantity for its mechanism, against five of six in the efficient-attention work the argument was borrowed from. We release the harness, the corrected implementation, the pre-registration and the five benchmarking pitfalls that produced spurious results along the way.

Broader Impact Statement

This work is a measurement and correction study of a computational-efficiency claim. It introduces no new capability and no new data. Its intended effect is to reduce wasted compute in the design of multi-agent

communication architectures by discouraging optimisation against an operation count that does not predict runtime. We note that a negative efficiency result of this kind is specific to the deployment regime studied (Section 7) and should not be read as an argument against sparse communication in bandwidth-constrained settings, where the objective is different.

References

Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. *ICML*, 2017. URL https://arxiv.org/abs/1705.10528.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint*, 2020. URL https://arxiv.org/abs/2004.05150.

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers. *ICLR 2021*, 2020. URL https://arxiv.org/abs/2009.14794.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Abhishek Das, Théophile Gervet, Joshua Romoff, Dhruv Batra, Devi Parikh, Michael Rabbat, and Joelle Pineau. Tarmac: Targeted multi-agent communication. *ICML*, 2019. URL https://arxiv.org/abs/1810.11187.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 2022. URL https://arxiv.org/abs/2101.03961.

Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. *ICLR*, 2017. URL https://arxiv.org/abs/1611.01144.

Jiechuan Jiang and Zongqing Lu. Learning attentional communication for multi-agent cooperation. *NeurIPS*, 2018. URL https://arxiv.org/abs/1805.07733.

Jiechuan Jiang, Chen Dun, Tiejun Huang, and Zongqing Lu. Graph convolutional reinforcement learning. *ICLR 2020*, 2018. URL https://arxiv.org/abs/1810.09202.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *ICLR*, 2020. URL https://arxiv.org/abs/2001.04451.

André F. T. Martins and Ramón Fernandez Astudillo. From softmax to sparsemax: A sparse model of attention and multi-label classification. *ICML*, 2016. URL https://arxiv.org/abs/1602.02068.

Georgios Papoudakis, Filippos Christianos, Lukas Schäfer, and Stefano V. Albrecht. Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks. *NeurIPS Datasets and Benchmarks*, 2021. URL https://arxiv.org/abs/2006.07869.

Ben Peters, Vlad Niculae, and André F. T. Martins. Sparse sequence-to-sequence models. *ACL*, 2019. URL https://arxiv.org/abs/1905.05702.

Markus N. Rabe and Charles Staats. Self-attention does not need $o(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *ICLR*, 2017. URL https://arxiv.org/abs/1701.06538.

Amanpreet Singh, Tushar Jain, and Sainbayar Sukhbaatar. Learning when to communicate at scale in multiagent cooperative and competitive tasks. *ICLR*, 2018. URL https://arxiv.org/abs/1812.09755.

Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by pid lagrangian methods. *ICML*, 2020. URL https://arxiv.org/abs/2007.03964.

Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. Learning multiagent communication with backpropagation. *NeurIPS*, 2016. URL https://arxiv.org/abs/1605.07736.

J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Rodrigo Perez, Caroline Horsch, Clemens Dieffendahl, Niall L. Williams, Yashas Lokesh, and Praveen Ravi. Pettingzoo: Gym for multi-agent reinforcement learning. *NeurIPS 2021*, 2020. URL https://arxiv.org/abs/2009.14471.

Muning Wen, Jakub Grudzien Kuba, Runji Lin, Weinan Zhang, Ying Wen, Jun Wang, and Yaodong Yang. Multi-agent reinforcement learning is a sequence modeling problem. *NeurIPS*, 2022. URL https://arxiv.org/abs/2205.14953.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of ppo in cooperative, multi-agent games. *NeurIPS Datasets and Benchmarks*, 2022. URL https://arxiv.org/abs/2103.01955.

Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. *NeurIPS*, 2020. URL https://arxiv.org/abs/2007.14062.

A Appendix

A.1 Dispersion behind every headline number

Table 3 reports min over interleaved samples, on the grounds that contention can only add time. That statistic is only trustworthy if the underlying sample was not wildly dispersed, so we report the median, the interquartile range and the spread of per-run medians for the same cells. A median close to the minimum indicates a quiet measurement; a large gap indicates the minimum is doing real work.

Table 7: Forward-pass dispersion, `fp32`, batch 256, idle device. Spread is the range of per-run medians across seven repeated timed blocks.

N	arm	min (ms)	median (ms)	IQR (ms)	median/min	spread
6	fused dense	0.1174	0.1248	0.0095	1.06	9.3%
	dense	0.1241	0.1303	0.0075	1.05	7.0%
	MASKED	0.1735	0.1813	0.0112	1.05	7.5%
	GATHERED	0.1457	0.1512	0.0061	1.04	2.2%
	gate	0.3513	0.3682	0.0297	1.05	11.0%
12	fused dense	0.1172	0.1213	0.0021	1.03	0.9%
	dense	0.1247	0.1284	0.0029	1.03	0.9%
	MASKED	0.1791	0.1837	0.0032	1.03	0.6%
	GATHERED	0.1507	0.1553	0.0030	1.03	0.4%
	gate	0.3613	0.3697	0.0075	1.02	1.1%

Across the full sweep the median-to-minimum ratio stays within 1.02–1.06×, so no headline number depends on an outlying fast sample. The complete table for all N and all arms, together with the raw per-run medians, is included in the released artifacts.

Table 8: Best-k gathered latency divided by unfused dense. Values < 1 would indicate sparsity winning; none do. "skip" marks cells whose activation memory exceeded the budget (at d=256, B=4096, N=512 the gathered value tensor alone is ≈273 GiB) and which are recorded as skipped rather than dropped.

d	batch	N=12	N=48	N=192	N=512
32	32	1.22	1.20	1.19	2.29
32	256	1.21	1.21	1.83	2.43
32	4096	1.35	1.98	1.48	2.36
64	32	1.22	1.22	1.22	2.35
64	256	1.23	1.22	1.76	2.46
64	4096	1.50	1.85	1.47	2.36
128	32	1.21	1.22	1.24	1.87
128	256	1.23	1.27	1.52	2.12
128	4096	1.45	1.91	1.50	skip
256	32	1.23	1.20	1.16	1.47
256	256	1.19	1.31	1.37	1.71
256	4096	1.28	1.61	1.45	skip

A.2 Crossover surface, tabulated

The numbers behind Figure 3.

A.3 Verification of the corrected implementation

The claim that GATHERED and MASKED compute the same function while executing different work is load-bearing, so it is pinned by regression tests rather than asserted:

- **Functional equivalence.** Maximum absolute deviation between the two paths is $< 10^{-10}$ in `float64` for $N \in \{6, 12, 24\}$ and $k \in \{1, 2, 3, N-1\}$.
- **Distinct executed work.** With N=48, k=4, B=8, d=16, a FLOP counter reports 2,555,904 for dense and MASKED (identical), against 2,015,232 for GATHERED. The difference is exactly the aggregation term, $2BNd(N-k)$.
- **No dense aggregation in the sparse path.** Tracing every `torch.matmul` confirms GATHERED issues no $(B, N, N) \times (B, N, d)$ product and does issue $(B, N, 1, k) \times (B, N, k, d)$.
- **The score matmul survives in all modes.** Every top-k variant still issues the full $(B, N, d) \times (B, d, N)$ product, which is assumption A2 of Proposition 1 made mechanical.
- **The ceiling holds numerically.** For $N \in \{6, \ldots, 512\}$ the ratio $C_{\text{dense}}/C_{\text{top-}k}$ is verified < 2, and the structural variant of Proposition 2 is verified to track N/k.

A.4 Reproduction

The measurement campaign is training-free and completes in minutes. The released harness regenerates every table in this paper from the raw JSON artifacts, and the training arms are driven by generated configuration files with the tier ordering enforced. The pre-registration document is committed with a timestamp preceding the unblinding of tiers 2–4, and its deviations log is empty apart from one recorded note concerning the bootstrap/permutation tension discussed in Section 5.